University of Debrecen
Faculty of Engineering
Mechatronics Engineering

Automated Conveyor Belt with Sensor-Based Colour Sorting

System Design Report

CodeAlpha Robotics & Automation Internship – Task 3

Prepared by
Mustafa Ali Mohsin Albusultan

1. Introduction

Conveyor belts are one of the most common elements of automated production and logistics. In their simplest form they only transport parts, but with a few sensors and a controller they can also count, inspect and sort them. This report presents the design of a small automated conveyor that transports objects, counts them, detects their colour and diverts red objects into a separate bin. The system must also stop safely when the emergency stop is pressed or when an object jams at the sensing station.

The design is presented as a flowchart of the control logic (Figure 1) and a circuit block diagram with the pin assignment (Figure 2). It is a design study: the system has not yet been built or tested, so the timing values given below are initial values that would be adjusted during commissioning.

2. System Overview

The belt is driven by a 12 V DC gear motor through an L298N motor driver, which allows the belt speed to be set by PWM. An infrared obstacle sensor (IR1) at the start of the belt registers every object that enters. Further along, a sensing station holds a second infrared sensor (IR2) and a TCS3200 colour sensor mounted above the belt. Directly after the station, a servo-driven diverter arm can swing across the belt: red objects are pushed into a side bin, while all other objects travel to a bin at the end of the belt. An Arduino Uno controls the system, and a 16x2 LCD shows the current state and the counts. The components and their roles are listed in Table 1.

Table 1. Main components of the conveyor system

Component	Function	Interface
Arduino Uno (ATmega328P)	Main controller; runs the control program	5 V logic, digital/analogue I/O
12 V DC gear motor	Drives the conveyor belt	Via L298N driver
L298N motor driver	Switches the motor and sets its speed	ENA (PWM), IN1, IN2
IR obstacle sensor 1	Detects objects entering the belt	Digital input
IR obstacle sensor 2	Detects objects at the sensing station	Digital input
TCS3200 colour sensor	Measures the colour of the object	Frequency output, S0–S3 select
MG90S servo + diverter arm	Pushes red objects into a side bin	PWM signal
16x2 LCD with I2C adapter	Shows state and object counts	I2C (SDA, SCL)
START (NO) / STOP (NC) buttons	Operator commands	Digital inputs, internal pull-up
Emergency stop, 2 NC contacts	Removes motor power; signals the controller	12 V motor line + digital input
Buzzer	Audible fault alarm	Digital output
12 V supply + 12 V/5 V buck converter	Power for motor (12 V) and electronics (5 V)	–

3. Control Logic

The control program is organised as three operating states: IDLE, RUN and FAULT (Figure 1). After power-on the controller initialises its inputs and outputs, switches the motor off, sets the diverter to its neutral position (0°) and resets the counters. In IDLE the belt is stopped and the LCD shows “Ready”. The belt is started only when the START button is pressed and the emergency stop is released.

In RUN the motor drives the belt at a fixed PWM duty cycle. In every program cycle the controller first checks the STOP button and the emergency stop; if either is active, the motor is switched off and the system returns to IDLE. Otherwise, a signal from IR1 increases the total count. When IR2 detects an object at the sensing station, the colour is read from the TCS3200. For a red object the diverter is moved to 45° for 1.5 s and the red count is increased; for any other colour the diverter stays at 0° and the second counter is increased.

A simple jam detection is included: if IR2 remains blocked for longer than 3 s, the object is assumed to be stuck. The controller then enters FAULT, stops the motor, switches on the buzzer and shows a message on the LCD. The system does not restart by itself; the operator has to clear the belt and press START, after which it returns to IDLE.

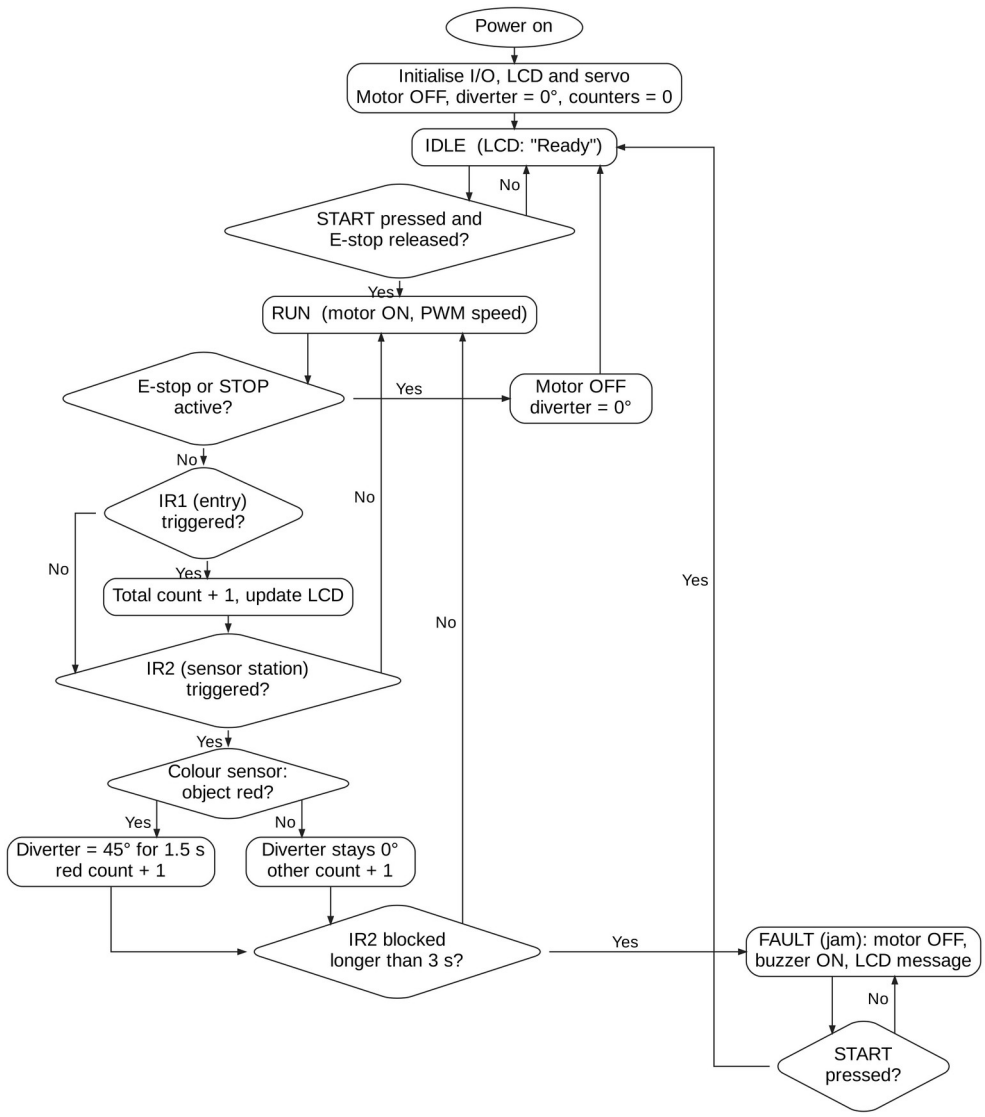


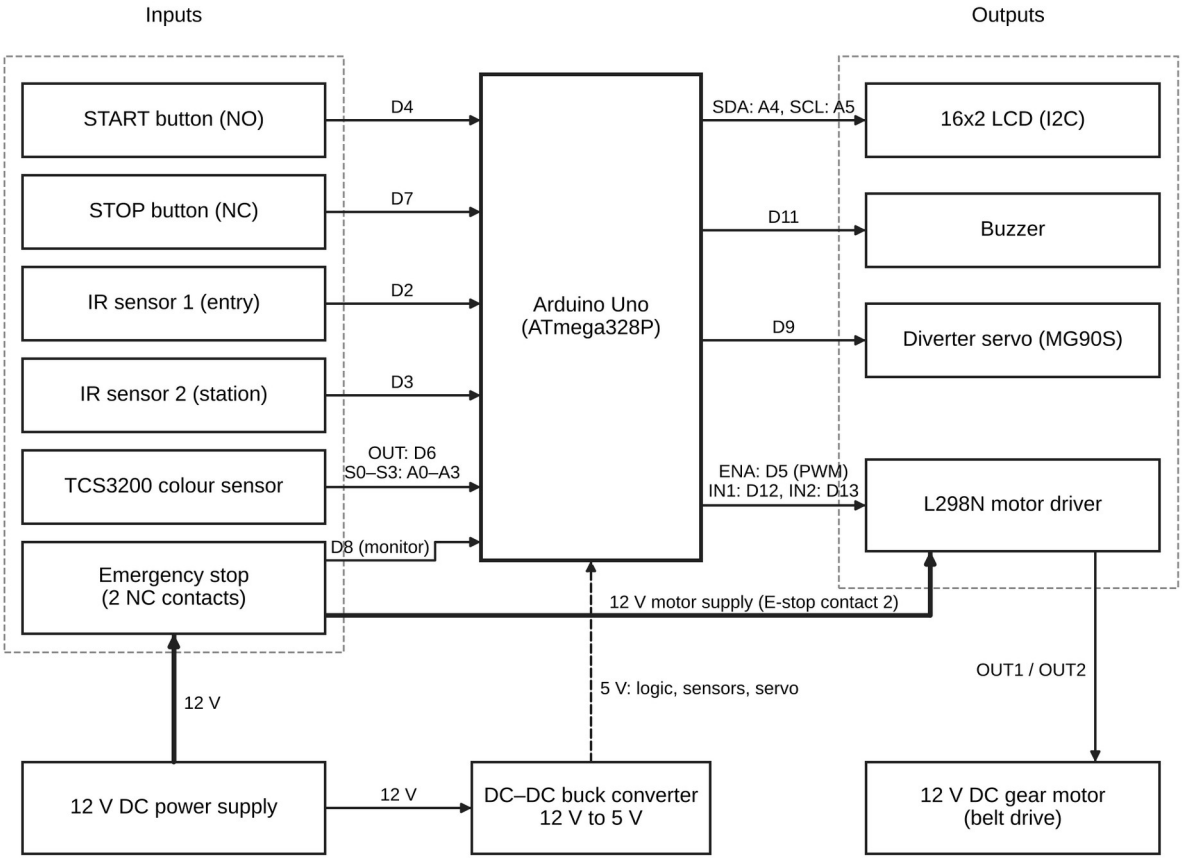
Figure 1. Flowchart of the conveyor control logic

4. Circuit Design

Figure 2 shows how the modules are connected. The two IR sensors are connected to D2 and D3, the two pins of the Arduino Uno that support external interrupts, so that short sensor pulses are not missed while the program is busy, for example while reading the colour sensor. The TCS3200 output frequency is measured on D6, and its filter and scaling selection pins S0–S3 use A0–A3 as digital outputs.

The motor driver enable input (ENA) is connected to D5. The Arduino Servo library uses Timer 1 and disables PWM on pins 9 and 10 [3], so the motor PWM is placed on a pin driven by a different timer, while the servo itself uses D9. The LCD uses the I2C pins A4 (SDA) and A5 (SCL), which keeps the wiring simple.

The STOP button and the emergency stop use normally closed (NC) contacts. With the internal pull-up resistors enabled, a pressed button or a broken wire produces the same input level, so a wiring fault stops the belt instead of preventing a stop. The motor is supplied with 12 V, and a buck converter provides 5 V for the Arduino, the sensors and the servo. All modules share a common ground.



All modules share a common ground (GND). The 12 V motor supply is interrupted by the E-stop independently of the software.

Figure 2. Circuit block diagram with the Arduino Uno pin assignment

5. Safety Considerations

The emergency stop has two NC contacts. The first is wired in series with the 12 V motor supply, so pressing it removes power from the motor directly, independently of the software. This corresponds to an uncontrolled stop by immediate removal of power, described as stop category 0 in IEC 60204-1 [2]. The second contact is read by the Arduino on D8, so the program also knows that the emergency stop is active and keeps the system in IDLE until it is released

and START is pressed again. Resetting the emergency stop alone does not restart the belt, in line with the general principles for emergency stop functions in ISO 13850 [1]. Together with the jam detection, this prevents the belt from continuing to run against a blocked object.

6. Conclusion

The proposed design combines simple and inexpensive components into a conveyor that counts, inspects and sorts objects automatically. The control logic is defined by three states with clear transitions, the pin assignment avoids timer conflicts between the motor PWM and the servo, and the emergency stop acts on the motor power in hardware. Possible extensions are sorting by more colours, logging the counts through an IoT module, or implementing the same logic on a PLC with ladder logic, which would be closer to industrial practice.

References

[1] ISO 13850:2015, *Safety of machinery – Emergency stop function – Principles for design*. Geneva, Switzerland: International Organization for Standardization, 2015.

[2] IEC 60204-1:2016, *Safety of machinery – Electrical equipment of machines – Part 1: General requirements*. Geneva, Switzerland: International Electrotechnical Commission, 2016.

[3] Arduino, “Servo library reference.” [Online]. Available: https://www.arduino.cc/reference/en/libraries/servo/